

YogiSpeaks - Setup & Run Guide

Complete guide to run the project with Docker or without Docker (local Node + PostgreSQL).

Communication coaching CMS - not a travel product.

1. Architecture (quick)

Service	Port	Role
Frontend (Next.js)	3000	Public site + Admin CMS UI
Backend (NestJS)	4000	REST API + Swagger + /uploads
PostgreSQL 16+	5432	Primary database
Redis 7	6379	Optional
Nginx (Docker only)	80	Reverse proxy

URL	Path
Site	http://localhost:3000
Admin login	http://localhost:3000/admin/login
API	http://localhost:4000/api/v1
Swagger	http://localhost:4000/api/docs
Health	http://localhost:4000/api/v1/health
Via Nginx (Docker)	http://localhost/

2. Prerequisites

Both paths

- Git
- Copy env templates - never commit real .env files

Without Docker

- Node.js 22+ (LTS)
- pnpm 10 (corepack enable && corepack prepare pnpm@10 --activate)
- PostgreSQL 16+ running locally
- Redis optional

With Docker

- Docker Desktop or Docker Engine + Compose plugin

3. Environment files

Three templates exist. Copy them once and fill real values locally.

```
cp .env.example .env
cp backend/.env.example backend/.env
cp frontend/.env.example frontend/.env.local
```

File	Used by
------	---------

.env	Docker Compose + shared defaults
backend/.env	NestJS when run with pnpm
frontend/.env.local	Next.js (NEXT_PUBLIC_*)

Values you must change

Variable	Notes
POSTGRES_PASSWORD / DATABASE_URL	Match user, password, DB name
JWT_ACCESS_SECRET / JWT_REFRESH_SECRET	>= 32 characters each
SUPER_ADMIN_EMAIL / SUPER_ADMIN_PASSWORD	Seed admin; password >= 12 chars
CORS_ORIGINS / FRONTEND_URL	Must match browser origin
NEXT_PUBLIC_API_URL	http://localhost:4000/api/v1

Example DATABASE_URL:

```
postgresql://yogispeaks:YOUR_PASSWORD@127.0.0.1:5432/yogispeaks?schema=public
```

Windows tip: prefer 127.0.0.1 over localhost if Postgres auth fails oddly.

4. Option A - Without Docker

Recommended for daily development.

4.1 Install PostgreSQL

1. Install PostgreSQL 16+ and start the service.
2. Create user/database:

```
CREATE USER yogispeaks WITH PASSWORD 'YOUR_PASSWORD' CREATEDB;  
CREATE DATABASE yogispeaks OWNER yogispeaks;
```

3. Put matching URL in backend/.env (and root .env if used).

Redis is optional.

4.2 Install dependencies

```
cd backend  
pnpm install  
  
cd ../frontend  
pnpm install
```

4.3 Migrate + seed

```
cd backend  
pnpm exec prisma generate  
pnpm exec prisma migrate deploy  
pnpm exec prisma db seed
```

Seed creates roles, permissions, and super-admin from SUPER_ADMIN_* in backend/.env.

4.4 Run backend + frontend (two terminals)

Terminal 1 - API:

```
cd backend  
pnpm start:dev
```

Terminal 2 - UI:

```
cd frontend
pnpm dev
```

4.5 Log in to Admin

Open <http://localhost:3000/admin/login>

Use SUPER_ADMIN_EMAIL / SUPER_ADMIN_PASSWORD from backend/.env

Uploads (local)

- MEDIA_PROVIDER=local
- Files in backend/uploads/ (auto-created)
- Gitignored - not pushed to GitHub
- Served at <http://localhost:4000/uploads/<filename>>

5. Option B - With Docker (full stack)

Compose starts: Postgres + Redis + Backend + Frontend + Nginx.

5.1 Configure root .env

```
cp .env.example .env
```

- Set POSTGRES_*, JWT_* (>=32), SUPER_ADMIN_* (>=12)
- NEXT_PUBLIC_SITE_URL=<http://localhost:3000>
- NEXT_PUBLIC_API_URL=<http://localhost:4000/api/v1>

Inside Docker network: DB host = postgres, Redis host = redis.

5.2 Build and start

```
docker compose up --build
# or detached:
docker compose up --build -d
```

5.3 Migrations + seed

```
docker compose exec backend pnpm exec prisma migrate deploy
docker compose exec backend pnpm exec prisma db seed
```

5.4 Open the apps

Entry	URL
Nginx (preferred)	http://localhost
Frontend direct	http://localhost:3000
Backend direct	http://localhost:4000/api/v1
Admin	http://localhost:3000/admin/login

5.5 Stop / reset

```
docker compose down
docker compose down -v # also wipes DB volume
```

Hybrid: Docker only for DB (+ Redis)

```
docker compose up postgres redis -d
```

Then run backend/frontend with Option A (pnpm). Expose ports 5432 and 6379 if needed.

6. Verify everything works

1. GET `http://localhost:4000/api/v1/health` -> ok
2. Swagger: `http://localhost:4000/api/docs`
3. Admin login with seeded super-admin
4. Upload in Admin -> Media; file appears under uploads/

7. Troubleshooting

Problem	Fix
Prisma P1000 / auth failed	Fix DATABASE_URL; use 127.0.0.1
citext errors	CREATE EXTENSION citext;
CORS / cookie issues	Match CORS_ORIGINS to browser origin
Env validation fails	JWT >=32 chars; admin pwd >=12
Admin 401	Re-seed; check NEXT_PUBLIC_API_URL
Docker backend unhealthy	docker compose logs backend
Uploads 404 (Nginx)	Proxy /uploads/ to backend
Port in use	Free 3000/4000/5432
pnpm not found	corepack prepare pnpm@10

8. Production notes

- NODE_ENV=production, strong secrets, COOKIE_SECURE=true, HTTPS
- Prefer Cloudinary/S3 over local disk when scaling
- Run prisma migrate deploy before serving traffic
- Never commit .env, uploads/, or node_modules/

9. Related docs

- docs/SETUP.md (markdown source)
- docs/IMPLEMENTATION_PLAN.md
- docs/DATABASE_PLAN.md
- docs/API_PLAN.md
- docs/TECHNOLOGY_DECISIONS.md